Question:

Given a binary tree, write a function that returns its maximum depth. The maximum depth is defined as the number of nodes along the longest path from the root node down to the farthest leaf node.

```
class Node:
    def __init__(self, value=None, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right

def max_depth(node):
    if not node:
        return 0
    left_depth = max_depth(node.left)
    right_depth = max_depth(node.right)
    return max(left_depth, right_depth) + 1

# Example usage:
root = Node(1, Node(2), Node(3, Node(4), Node(5)))
print(max_depth(root))  # Output: 3
```

Explanation:
The `max_depth` function recursively calculates the maximum depth of a binary tree. The base case is when the current node is `None`, in which case the function returns 0. For any other node, it calculates the maximum depth of its left and right subtrees recursively, and returns the maximum of those depths plus 1 (to include the current node in the count). The example usage creates a sample binary tree and prints its maximum depth using the `max_depth` function.

Write a program to implement a stack in JavaScript.

1. Answer:

```
class Stack { constructor() { this.items = []; } push(element) {
this.items.push(element); } pop() { if (this.items.length === 0) { return
```

```
"Underflow"; } return this.items.pop(); } peek() { return
this.items[this.items.length - 1]; } isEmpty() { return this.items.length === 0; }
printStack() { let str = ""; for (let i = 0; i < this.items.length; i++) { str +=
this.items[i] + " "; } return str; } } const stack = new Stack(); stack.push(10);
stack.push(20); stack.push(30); console.log(stack.printStack()); // output: 10 20
30 console.log(stack.pop()); // output: 30 console.log(stack.peek()); // output:
20
```

Implement a program that sorts an array of integers in ascending order using the bubble sort algorithm in Python.

2. Answer:

less

Copy code

```
def bubbleSort(arr): n = len(arr) for i in range(n): for j in range(0, n-i-1): if
arr[j] > arr[j+1] : arr[j], arr[j+1] = arr[j+1], arr[j] return arr arr = [64, 34,
25, 12, 22, 11, 90] print(bubbleSort(arr)) # output: [11, 12, 22, 25, 34, 64, 90]
```

Write a program in TypeScript to find the largest element in an array.

3. Answer:

arduino

Copy code

```
function findLargestElement(arr: number[]): number { let max = arr[0]; for (let i
= 1; i < arr.length; i++) { if (arr[i] > max) { max = arr[i]; } } return max; }
const arr = [10, 50, 20, 100, 30]; console.log(findLargestElement(arr)); //
output: 100
```

Implement a program in Node.js that reads a file and prints its contents to the console.

4. Answer:

```
const fs = require('fs'); fs.readFile('file.txt', 'utf8', function(err, data) { if
(err) throw err; console.log(data); });
```

Write a program in Python to find the first non-repeating character in a string.

5. Answer:

rust

Copy code

```
def findFirstNonRepeatingChar(str): char_count = {} for char in str: if char in
char_count: char_count[char] += 1 else: char_count[char] = 1 for char in str: if
char_count[char] == 1: return char return None str = "abacabad"
print(findFirstNonRepeatingChar(str)) # output: c
```

1. Write a function that finds the sum of all even numbers in an array of integers.

```
function sumEvenNumbers(arr) {
 let sum = 0;
 for (let i = 0; i < arr.length; i++) {
  if (arr[i] % 2 === 0) {
   sum += arr[i];
  }
 }
 return sum;
}

// Example usage
const arr = [1, 2, 3, 4, 5, 6];
console.log(sumEvenNumbers(arr)); // Output: 12
```

2. Write a function that takes a string and returns a new string with all vowels removed.

```
function removeVowels(str) {
 const vowels = "aeiouAEIOU";
 let result = "";
 for (let i = 0; i < str.length; i++) {
  if (!vowels.includes(str[i])) {
   result += str[i];
  }
 }
```

```
  return result;
}

// Example usage
const str = "hello world";
console.log(removeVowels(str)); // Output: "hll wrld"
```

3. Write a function that determines if a string is a palindrome.

```
function isPalindrome(str) {
  str = str.toLowerCase().replace(/[^a-zA-Z0-9]/g, "");
  const reversedStr = str.split("").reverse().join("");
  return str === reversedStr;
}

// Example usage
const str1 = "A man, a plan, a canal, Panama!";
const str2 = "race car";
console.log(isPalindrome(str1)); // Output: true
console.log(isPalindrome(str2)); // Output: true
```

4. Write a function that merges two sorted arrays into a single sorted array.

```
function mergeSortedArrays(arr1, arr2) {
  const result = [];
  let i = 0;
  let j = 0;
  while (i < arr1.length && j < arr2.length) {
    if (arr1[i] < arr2[j]) {
      result.push(arr1[i]);
      i++;
    } else {
      result.push(arr2[j]);
      j++;
    }
  }
  return result.concat(arr1.slice(i)).concat(arr2.slice(j));
```

```
}

// Example usage
const arr1 = [1, 3, 5];
const arr2 = [2, 4, 6];
console.log(mergeSortedArrays(arr1, arr2)); // Output: [1, 2, 3, 4, 5, 6]
```

5. Write a function that checks if a given number is a prime number.

```
function isPrime(num) {
  if (num <= 1) {
    return false;
  }
  for (let i = 2; i <= Math.sqrt(num); i++) {
    if (num % i === 0) {
      return false;
    }
  }
  return true;
}

// Example usage
console.log(isPrime(7)); // Output: true
console.log(isPrime(12)); // Output: false
```

6. Can you explain the difference between synchronous and asynchronous programming in Node.js?

Answer: Synchronous programming blocks the execution of code until a particular task is finished, while asynchronous programming allows multiple tasks to be executed at the same time. In Node.js, asynchronous programming is often used for I/O-bound tasks to avoid blocking the event loop.

7. Can you explain what the event loop is in Node.js?

Answer: The event loop is a key feature of Node.js that enables asynchronous programming. It is responsible for handling all the events and callbacks in the Node.js application. The event loop constantly checks for any new events or callbacks to be

executed, and when an event or callback is detected, it is added to the event queue for execution.

8. How would you handle errors in a Node.js application?

Answer: In a Node.js application, errors can be handled using try-catch blocks or error handling middleware functions. Additionally, logging frameworks like Winston or Bunyan can be used to log errors to a file or database for analysis and debugging.

9. Can you explain how Node.js handles module dependencies?

Answer: Node.js uses the CommonJS module system to handle module dependencies. Modules can be defined using the require() function, and can be exported using the module.exports or exports objects. When a module is required, Node.js loads the module code into memory and caches it, so subsequent calls to require() will return the cached module.

10. How would you optimize the performance of a Node.js application?

Answer: There are several ways to optimize the performance of a Node.js application, including:

- Using a caching layer to reduce the number of database queries
- Using compression to reduce the size of HTTP responses
- Using a load balancer to distribute requests across multiple servers
- Minimizing the number of blocking I/O operations
- Using a process manager like PM2 to manage multiple Node.js processes
- Using a performance monitoring tool like New Relic or AppDynamics to identify and fix performance issues.

11. Implement a function that takes a string as input and returns the first non-repeating character in the string. (Node.js)

```
function findFirstNonRepeatingCharacter(str) {
  const charCount = {};
  for (let char of str) {
    charCount[char] = charCount[char] + 1 || 1;
  }
  for (let char of str) {
```

```
    if (charCount[char] === 1) {
      return char;
    }
  }
  return null;
}
```

12. Write a program that sorts an array of integers in ascending order using the bubble sort algorithm. (Node.js)

```
function bubbleSort(arr) {
  let swapped;
  do {
    swapped = false;
    for (let i = 0; i < arr.length - 1; i++) {
      if (arr[i] > arr[i + 1]) {
        [arr[i], arr[i + 1]] = [arr[i + 1], arr[i]];
        swapped = true;
      }
    }
  } while (swapped);
  return arr;
}
```

13. Implement a function that checks whether a given string is a palindrome. (Node.js)

```
function isPalindrome(str) {
  const reversed = str.split("").reverse().join("");
  return str === reversed;
}
```

14. Write a program that takes an array of integers as input and returns the second largest integer in the array. (Node.js)

```
function findSecondLargest(arr) {
  let largest = arr[0];
  let secondLargest = null;
  for (let i = 1; i < arr.length; i++) {
```

```
    if (arr[i] > largest) {
      secondLargest = largest;
      largest = arr[i];
    } else if (arr[i] < largest && (secondLargest === null || arr[i] > secondLargest)) {
      secondLargest = arr[i];
    }
  }
  return secondLargest;
}
```

15. Implement a function that filters an array of objects based on a given key-value pair. (Node.js)

```
function filterByValue(arr, obj) {
  return arr.filter(item => {
    for (let key in obj) {
      if (item[key] !== obj[key]) {
        return false;
      }
    }
    return true;
  });
}
```

16. Write a function that takes a string and returns a new string with the letters reversed. (JavaScript)

```
function reverseString(str) {
  return str.split('').reverse().join('');
}

console.log(reverseString('hello')); // olleh
```

17. Write a function that takes an array of numbers and returns the sum of all the even numbers. (TypeScript)

```
function sumEvenNumbers(numbers: number[]): number {
```

```
  return numbers.filter(num => num % 2 === 0).reduce((acc, curr) => acc + curr, 0);
}

console.log(sumEvenNumbers([1, 2, 3, 4, 5, 6])); // 12
```

18. Implement a function that finds the maximum subarray sum of a given array.
Node.js Solution:

```
function maxSubarraySum(arr) {
  let maxSum = 0;
  let currentSum = 0;

  for (let i = 0; i < arr.length; i++) {
    currentSum += arr[i];
    if (currentSum > maxSum) {
      maxSum = currentSum;
    }
    if (currentSum < 0) {
      currentSum = 0;
    }
  }

  return maxSum;
}
```

19. Implement a function that finds the intersection of two arrays.

```
function intersection(arr1, arr2) {
  const set1 = new Set(arr1);
  const set2 = new Set(arr2);

  return [...set1].filter(item => set2.has(item));
}
```

20. Implement a function that flattens a nested array.

```
function flattenArray(arr) {
  return arr.reduce((flatArray, item) => {
    return flatArray.concat(Array.isArray(item) ? flattenArray(item) : item);
  }, []);
}
```